

Simple Scanner

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

#include<ctype.h> // Include ctype.h for isalpha and isdigit functions


char
kw[32][10]={ "int","float","while","for","do","char","break","auto","continue",

"default","double","if","else","enum","goto","long","switch","typedef","union",
            "unsign","void","volatile","extern","case","const"};

char op[15]={'+','-','*','/','=',':',';','<','>'};

char ip[15];

char identifiers[20];

char constants[20];

char operators[20];

int oc=0,cc=0,ic=0;


void main()
{
    char str[20];
    char *ptr;
    int i=0,j;

    clrscr();

    printf("\n Enter String");

    scanf("%[^\\n]s",str);

    ptr=strtok(str," ");
```

```

printf("Keyword is:- ");
while(ptr!=NULL)
{
    int flag=0;
    for(i=0;i<32;i++)
    {
        if(strcmp(ptr,kw[i])==0)
        {
            printf("%s , ",ptr);
            flag=1;
        }
    }
    if(flag==0)
    {
        strcat(ip,ptr);
    }
    ptr= strtok(NULL," ");
}
for(i=0;i<strlen(ip);i++)
{
    if(isalpha(ip[i]))
    {
        identifiers[ic] = ip[i];
        ic++;
    }
    else if(isdigit(ip[i]))
    {

```

```

        constants[cc]=ip[i];
        cc++;
    }
    else
    {
        for(j = 0; j < sizeof(op); j++)
        {
            if(ip[i] == op[j])
            {
                operators[oc] = ip[i];
                oc++;
                break; // Exit the loop once the operator is found
            }
        }
    }
}

printf("\n Identifiers : ");
for(i=0;i<ic;i++)
{
    printf("%c ",identifiers[i]);
}

printf(" \nConstants : ");
for(i=0;i<cc;i++)
{
    printf("%c ",constants[i]);
}

printf(" \nOperators : ");

```

```

    for(i=0;i<oc;i++)
    {
        printf("%c ", operators[i]);
    }
    getch();
}

```

Scanner with dynamic table.

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<ctype.h>

char
kw[32][10]={"int","float","while","for","do","char","break","auto","continue",

"default","double","if","else","enum","goto","long","switch","typedef","union",
           "unsign","void","volatile","extern","case","const"};

char op[15]={'+','-','*','/','=',':',';','<','>'};
char ip[100];
char identifiers[100];
char constants[100];
char operators[100];
int oc=0,cc=0,ic=0;

void main()
{

```

```

char str[100];
char *ptr;
int i=0, j=0, flag=0; // Declare loop variables and flags

clrscr();
printf("\n Enter String");
scanf("%[^\\n]s",str);
ptr=strtok(str, " ");
printf("Keyword is:- ");
while(ptr!=NULL)
{
    flag=0;
    for(i=0;i<32;i++)
    {
        if(strcmp(ptr,kw[i])==0)
        {
            printf("%s , ",ptr);
            flag=1;
        }
    }
    if(flag==0)
    {
        strcat(ip,ptr);
    }
    ptr=strtok(NULL, " ");
}
for(i=0;i<strlen(ip);i++)

```

```

{
    if(isalpha(ip[i]))
    {
        // Check if the identifier already exists
        int exists = 0;
        for(j = 0; j < ic; j++) {
            if(ip[i] == identifiers[j]) {
                exists = 1;
                break;
            }
        }
        if(!exists) {
            identifiers[ic] = ip[i];
            ic++;
        }
    }
    else if(isdigit(ip[i]))
    {
        // Check if the constant already exists
        int exists = 0;
        for(j = 0; j < cc; j++) {
            if(ip[i] == constants[j]) {
                exists = 1;
                break;
            }
        }
        if(!exists) {

```

```

        constants[cc] = ip[i];
        cc++;
    }
}
else
{
    for(j = 0; j < sizeof(op); j++)
    {
        if(ip[i] == op[j])
        {
            operators[oc] = ip[i];
            oc++;
            break;
        }
    }
}

printf("\n Identifiers : ");
for(i=0;i<ic;i++)
{
    printf("%c ",identifiers[i]);
}

printf(" \nConstants : ");
for(i=0;i<cc;i++)
{
    printf("%c ",constants[i]);
}

```

```
printf(" \nOperators : ");
for(i=0;i<oc;i++)
{
    printf("%c ", operators[i]);
}

printf("\n\nDynamic Table of Constants and Identifiers:\n");
printf("Identifier\n");
for(i=0; i<ic; i++)
{
    printf("%c\n", identifiers[i]);
}
printf("Constant\n");
for(i=0; i<cc; i++)
{
    printf("%c\n", constants[i]);
}
getch();
}
```